

Professional Coding Specialist

COS Pro 파이썬 2 급

21 강-23 강. 모의고사 5 차

모의고사 5 차 문제 1-10 번

과정 소개

모의고사(YBM 샘플) 문제를 통해 실전에 대비할 수 있다. (회차당 10 문항, 각 문항별 문제 풀이)

학습 목차

모의고사 5 차 문제 1 번
모의고사 5 차 문제 2 번
모의고사 5 차 문제 3 번
모의고사 5 차 문제 4 번
모의고사 5 차 문제 5 번
모의고사 5 차 문제 6 번
모의고사 5 차 문제 7 번
모의고사 5 차 문제 8 번
모의고사 5 차 문제 9 번
모의고사 5 차 문제 10 번

학습 목표

1. YBM(www.ybmit.com)에서 제공하는 COS Pro 파이썬 2 급 샘플 문제 메뉴의 모의고사를 풀어보며 COS Pro 파이썬 2 급 시험에 대비할 수 있다.

1

모의고사 5 차 문제 1 번

문제 1 의 정답

사다리 게임 당첨자 찾기

```
def solution(ladders, win):
    answer = 0
    player = [1, 2, 3, 4, 5, 6]
    ① for e in ladders:
        ② temp = player[e[0]-1]
        player[e[0]-1] = player[e[1]-1]
        player[e[1]-1] = temp
    ③ answer = player[win-1]
    return answer
```

	ladders	win	return
	[[1, 2], [3, 4], [2, 3], [4, 5], [5, 6]]	3	1

e [0] [1]
e [1, 2]
 temp = a
 a = b
 b = temp

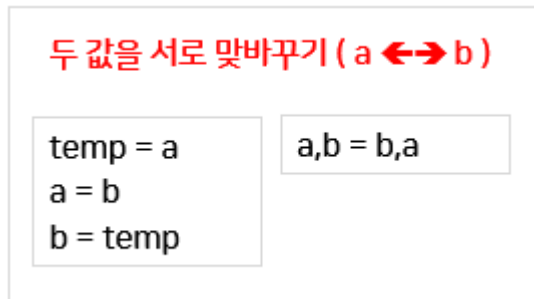
시작 위치
 도착 번호
 변화

	0	1	2	3	4	5
1	2	3	4	5	6	
2	1	3	4	5	6	
2	1	4	3	5	6	
2	4	1	3	5	6	
2	4	1	5	3	6	
2	4	1	5	6	3	

- ① 2 차원 리스트에 있는 1 차원 리스트 형태로 가로줄의 데이터를 하나씩 읽어 와서
- ② [가로줄 시작 번호, 가로줄 종료 번호] 인덱스에 있는 값을 서로 맞바꾼다. 가져온 값은 사다리 게임의 가로줄 시작 번호는 e[0]을, 가로줄 종료 번호는 e[1]을 이용한다.
- ③ 가져온 win 변수에 있는 값은 3 이므로 2 번 인덱스의 값(당첨자 번호)을 리턴한다.

문제의 형태는 사다리 게임이지만, 실제 코드는 두 수의 값을 서로 맞바꾸는 문제이다.

두 값을 맞바꿀 때는 아래 왼쪽 그림처럼 temp 라는 임시 변수를 이용해서 바꿀 수도 있지만, 파이썬에서는 a,b=b,a 처럼 한 문장으로 두 값을 맞바꿀 수 있다.



[2 급 5 차 1 번 다른 코딩 제안.py]

아래는 두 값을 a,b=b,a 처럼 한 문장으로 맞바꾼 예시이다.

```
def solution(ladders, win):
    answer = 0
    player = [1, 2, 3, 4, 5, 6]

    for e in ladders:
        player[e[0]-1], player[e[1]-1] = player[e[1]-1], player[e[0]-1]

    answer = player[win-1]
    return answer
```

2

모의고사 5 차 문제 2 번

문제 2 의 정답

총 공장 시간 구하기

```

def func_a(time_table):
    2 answer = 0
    for i, t in reversed(list(enumerate(time_table))):
        if t == 1:
            answer = i
            break
    return answer

def func_c(time_table):
    1 answer = 0
    for i, t in enumerate(time_table):
        if t == 1:
            answer = i
            break
    return answer

def func_b(time_table, class1, class2):
    3 answer = 0
    for i in range(class1, class2):
        if time_table[i] == 0:
            answer += 1
    return answer

def solution(time_table):
    answer = 0
    first_class = func_c(time_table)
    last_class = func_a(time_table)
    answer = func_b(time_table, first_class, last_class)
    return answer

```

인덱스	0	1	2	3	4	5	6	7	8	9	9	8	7	6	5	4	3	2	1	0
공장여부	1	1	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	0	1	1

1. 시작 시각 찾기 2. 종료 시각 찾기

① 첫 수업 시작 시각을 구한다.

<func_c 함수>

```
def func_c(time_table):  
    answer = 0  
    for i, t in enumerate(time_table):  
        if t == 1:  
            answer = i  
            break  
    return answer
```

처음 1 을 만나면 해당 인덱스를 리턴함으로써 수업을 시작하는 시각을 구한다. enumerate() 함수를 이용하여 인덱스 번호(i)와 시간(t)을 가져와 시간(t)이 1 이면 해당 인덱스 번호를 리턴하고, break 문장을 이용해서 빠져나간다. break 문장을 이용함으로써 처음 1 을 만난 인덱스 번호를 리턴할 수 있다.

② 마지막 수업 시작 시각을 구한다.

<func_a 함수>

```
def func_a(time_table):  
    answer = 0  
    for i, t in reversed(list(enumerate(time_table))):  
        if t == 1:  
            answer = i  
            break  
    return answer
```

리스트의 인덱스와 데이터를 역순으로 정렬한 후 처음 만난 인덱스 번호를 리턴하는 방식으로 마지막 수업 시각을 구한다.

③ ①과 ②사이에서 수업이 없는 시간을 센다.

<func_b 함수>

```
def func_b(time_table, class1, class2):  
    answer = 0  
    for i in range(class1, class2):  
        if time_table[i] == 0:  
            answer += 1  
    return answer
```

이 문제에서는 수업이 끝난 후의 0 은 공간이 아니라는 점에 주의해야 한다. 따라서 수업 시작 시간과 수업 종료 시간 사이에 있는 0 의 개수만 세야 함에 주의한다.

[2 급 5 차 2 번 다른 코딩 제안.py]

```
def func_c(time_table):  
    answer = 0  
    for i in range(len(time_table)):  
        if time_table[i] == 1:  
            answer = i  
            break  
    return answer
```

1. 시작 시각 찾기

```
def func_a(time_table):  
    answer = 0  
    for i in range(len(time_table)-1,-1,-1):  
        if time_table[i] == 1:  
            answer = i  
            break  
    return answer
```

2. 종료 시각 찾기

```
def func_b(time_table, class1, class2):  
    answer= time_table[class1:class2].count(0)  
    return answer
```

3. 시작에서 종료 시각 사이의 공강의 개수 구하기

※ 시작 시각과 종료 시각을 찾을 때 enumerate()함수를 사용하지 않고 인덱스 방식을 사용한 예제이다. 종료 시각을 찾을 때 맨 뒤에서부터 시작해서 맨 앞까지 인덱스 번호를 옮기면서 가장 먼저 1 이 나올 때를 찾으면 정렬하지 않아도 된다.

또한 시작에서 종료 시각 사이의 공강의 개수를 구할 때 time_table[class1:class2]와 같이 문자열 슬라이싱 방식을 이용해서 시작 시각에서 종료 시각까지의 데이터만 잘라온다. 잘라온 후에 이 데이터는 리스트 형태이므로 리스트의 count() 메소드를 사용하여 0 의 개수를 세면 쉽게 답을 구할 수 있다.

3

모의고사 5 차 문제 3 번

문제 3 의 정답

총 벌금 계산

```
def solution(speed, cars):
    answer = 0
    for x in cars:
        if x >= speed * 11 / 10 and x < speed * 12 / 10:
            answer += 3
        elif x >= speed * 12 / 10 and x < speed * 13 / 10:
            answer += 5
        elif x >= speed * 13 / 10:
            answer += 7
    return answer
```

차량들의 속도가 담긴 cars 리스트를 가져와서 속도 10% 이상 20% 미만은 벌금 3만원, 20% 이상 30% 미만은 벌금 5만원, 속도 30% 이상은 7만원의 벌금을 부과한다.

4

모의고사 5 차 문제 4 번

문제 4 의 정답

선수의 총 점수 계산

```
def solution(taekwondo, running, shooting):
    answer = 0
    if taekwondo >= 25:
        answer += 250
    else:
        answer += taekwondo * 8
    answer += 250 + (60 - running) * 5
    count = 0
    for s in shooting:
        answer += s
        if s == 10:
            count += 1
    if count >= 7:
        answer += 100
    return answer
```

종목	점수 산출 방식
1 태권도	25경기 이상 승리하면 250점 그 외에는 승리당 8점
2 500m 달리기	60초에 완주 시 250점 그보다 빠르면 1초당 +5점 느리면 1초당 -5점
3 사격	10번 사격 과녁에 적힌 숫자의 합만큼 점수 획득 7번 이상 10점에 맞추면 추가 점수 100점

점수 산출 방식에 맞게 코드한다.

문제 5 의 정답

두 개의 장이 동시에 열리는 날 구하기

```
def solution(a, b):
    answer = 0
    ① for i in range(1, b + 1):
        ② if (a * i) % b == 0:
            answer = a * i
            break
    return answer
```

이 문제는 두 수의 최소공배수를 구하는 문제이다.

① i가 1 부터 시작해서 b 까지 돌면서

② $(a * i)$ 를 b로 나눈 나머지가 0 이면 $a * i$ 가 공배수이다.

이 공배수 중 가장 첫 번째 것이 최소공배수이므로 break 를 사용한다.

약수, 공약수, 최대공약수를 구하는 방법과 배수, 공배수, 최소공배수를 구하는 알고리즘을 정리한다.

약수, 공약수, 최대공약수

- 약수 : 어떤 수를 나누어 떨어지게 하는 수, 1 과 자기 자신은 모두 약수
- 공약수 : 두 수의 공통되는 약수
- 최대공약수 : 공약수 중 가장 큰 것
- ※ 두 수의 공약수는 작은 수를 넘을 수 없다.

배수, 공배수, 최소공배수

- 배수 : 2 의 배수는 2 로 나누어 떨어지는 수
- 공배수 : 두 수의 공통되는 배수
- 최소공배수 : 공배수 중 가장 작은 것
- ※ 2 와 4 의 최소공배수는 두 수의 곱(2×4) 이하의 숫자이다.

최소공배수 구하는 법-1

[알고리즘 최소공배수-1.py]

```
def solution(a,b):
    answer=0
    for i in range(1,a*b+1):
        if i%a==0 and i%b==0:
            answer=i
            break
    return answer

v=solution(4,6)
print(v)
```

최소공배수 구하는 법-2

[알고리즘 최소공배수-2.py]

```
def solution(a,b):
    answer=0
    for i in range(1,b+1):
        if (a*i)%b==0:
            answer=a*i
            break
    return answer

v=solution(4,6)
print(v)
```

6

모의고사 5 차 문제 6 번

문제 6 의 정답

수학 점수의 최솟값 구하기

```
def solution(korean, english):  
    answer = 0  
    math = 210 - (korean + english)  
    if math > 100:  
        answer = -1  
    else:  
        answer = math  
    return answer
```

세 과목의 평균이 70 을 넘기 위해 받아야 하는 수학 점수를 구하는 문제로 $210 - (\text{국어 점수} + \text{영어 점수})$ 의 계산식을 사용해야 한다. 여기서는 괄호가 생략되었다.

문제 7 의 정답

물건 계산에 걸리는 시간 구하기

```
def solution(stuffs):
    answer = 0
    ① small_counter, general_counter = 0, 0
    ② for s in stuffs:
        if s > 3:
            general_counter += s
        else:
            small_counter += s
    ③ if small_counter > general_counter:
        answer = small_counter
    else:
        answer = general_counter
    return answer
```

stuffs	return
[5, 3, 4, 2, 3, 2]	10

일반 계산대

소량 계산대

0	1
5	4

0	1	2	3
3	2	3	2

- ① 각 계산대에서 걸리는 시간을 0 으로 초기화 한다.
마트에서 소량 계산대, 일반 계산대로 나눠서 계산이 진행되므로 소량 계산대(small_counter)와 일반 계산대(general_counter) 변수를 각각 0 으로 초기화 한다.
- ② 각 고객의 물건의 개수가 담긴 stuffs 데이터를 하나씩 읽어 와서 물건의 개수가 3 보다 크면 일반 계산대에서, 아니면 소량 계산대에서 계산하는 시간에 합한다.
- ③ 두 계산대에서 계산이 동시에 진행되므로 계산대와 소량 계산대 중 더 오래 걸리는 곳의 시간을 리턴하면 총 소요되는 시간을 구하게 된다.

8

모의고사 5 차 문제 8 번

문제 8 의 정답

상수도 요금 계산

```
def solution(usage):
    answer = 0
    if usage > 30:
        answer = 20 * 430 + 10 * 570 + (usage - 30) * 840
    elif usage > 20:
        answer = 20 * 430 + (usage - 20) * 570
    else:
        answer = usage * 430
    return answer
```

단계	사용량	요금
1단계	0~20톤	430원
2단계	21~30톤	570원
3단계	31톤 이상	840원

1단계 적용 : 20톤 x 430원 = 8,600원
 2단계 적용 : 10톤 x 570원 = 5,700원
 3단계 적용 : 5톤 x 840원 = 4,200원
 총 사용요금 : 18,500원

이 문제에서는 상수도 사용량에 따른 요금을 책정하되 사용량마다 1 톤당 부과되는 금액이 다른 예제이다.

단계별 적용 요금을 참고해서 계산식에 오류가 없는지 찾는다.

문제 9 의 정답

점수 순위 구하기

[2 급 5 차 9 번 다른 코딩 제안-1.py]

```
def solution(score):
    answer = []
    ① for i in range(len(score)):
        ② rank=1
        ③ for s in score:
            if score[i] < s:
                rank+=1
        ④ answer.append(rank)
    return answer
```

0	1	2	3	0	1	2	3
10	20	20	30	4	2	2	1

score 리스트 : 점수 answer 리스트 : 순위

- ① 점수가 담긴 score 리스트의 인덱스를 0 부터 끝까지 돌면서 순위를 구할 기준을 score[i]로 정한다.
- ② 순위(rank)를 무조건 1 위로 초기화한다.
- ③ 현재 순위를 구하고자 하는 기준 데이터 score[i] 와 나머지 데이터를 모두 비교해서 현재(score[i])보다 큰 값이 있으면 순위를 하나씩 증가하는 방식으로 score[i]의 순위를 구한다.
- ④ 구해진 순위(rank)를 answer 리스트에 append 함으로써 모든 점수의 순위를 구한 리스트를 반환한다.

[2 급 5 차 9 번 다른 코딩 제안-2.py]

```
def solution(scores):
    answer=[1]*len(scores)

    for i in range(len(scores)):
        for j in range(len(scores)):
            if scores[i] < scores[j]:
                answer[i]+=1

    return answer
```

[5 차 2 급 9_solution_code.py]

```
def solution(score):
    answer = [0] * len(score)
    for i in range(len(score)):
        answer[i] = sum(map(lambda x:x > score[i], score))+1
    return answer
```

※ 9 번 정답지에 있는 코드에는 람다와 map 을 사용하고 있음

lambda : 함수 이름에 대한 정의 없이 간단히 사용하는 함수

```
#람다 함수식
def func(a,b):
    return a+b
print(func(10,20))

s1=lambda a,b:a+b
print(s1(10,20))
```

lambda 매개 변수 : 리턴 값

map : 리스트의 항목마다 앞의 함수를 적용해 주는 함수

```
a=['1','2','3','4','5']

a=map(int,a)
print(list(a))
```


10

모의고사 5 차 문제 10 번

문제 10의 정답

가장 오래 근무한 시간 구하기

```
def solution(time_table, n):
    #여기에 코드를 작성해주세요.
    answer = 0
    1 work=[0]*n
    2 for i, t in enumerate(time_table):
        work[i%n]+=t
    3 answer=max(work)
    return answer
```

time_table				사람별 총 근무 시간		
0	1	2	3	0	1	2
1	5	1	9	1	5	1
				9		

인덱스 번호: time_table의 인덱스
번호를 n으로 나눈 나머지

- ① 각 사람별로 일한 시간을 저장할 수 있는 리스트를 초기화한다.
- ② time_table의 데이터를 하나씩 읽어 와서 각 개인별 총 근무 시간을 구한다.
time_table을 enumerate() 함수를 이용하여 인덱스와 데이터를 함께 가져온다.
인덱스 번호를 n으로 나눈 나머지가 work의 인덱스 번호로 사용하는 것에 주의한다.
- ③ 근무 시간 중 최댓값을 찾는다.